

Two Approaches to GenAI Inference: IREE's MLIR Compiler Stack and Llama.cpp

Adeel Ahmad, Taimur Ahmad
10xEngineers Inc.

Namal Summer School | September 5, 2026

Speaker's Introduction

Meet today's presenters.



Taimur Ahmad

Senior Compiler Engineer · 10xEngineers

Working on high-performance AI inference for RISC-V-based hardware. Optimizing AI compute kernels for RISC-V and other custom RISC-V-based hardware. Contributed towards the development of RVV kernels in upstream Llama.cpp.

Llama.cpp

AI kernels



Adeel Ahmad

Senior Compiler Engineer · 10xEngineers

Working on optimizing the compilation of LLMs for RISC-V CPUs and RISC-V-based custom hardware in MLIR-based AI compiler. Experienced in writing optimized kernels, LLVM middle-end and backend development.

MLIR

IREE

Contents

- **Introduction to LLVM & MLIR**
 - *LLVM's Three-Phase Flow: Too Low-Level for AI*
 - *MLIR: A Framework to Build IRs*
 - *MLIR for AI Workloads*
- **IREE Compilation Pipeline for RISC-V**
- **Introduction to Llama.cpp**
 - *Llama.cpp pipeline: GGML and GGUF*
 - *Quantization Formats*
 - *Compute Kernels for RISC-V*
- **Comparing IREE and Llama.cpp for AI Inference**

LLVM's Three-Phase Flow: Too Low-Level for AI



The frontend / optimizer / backend split is brilliant => but it pivots on a single, machine-level IR.



WHAT LLVM IR WORKS WITH

loads · stores · add · fmul · branches · vectors ops

One level of abstraction + close to the metal.

WHAT AN AI MODEL NEEDS

tensors · matmul · conv · attention · loop nests · fusion ·
tiling · layout

*High-level structure, already gone by the time you reach
LLVM IR.*

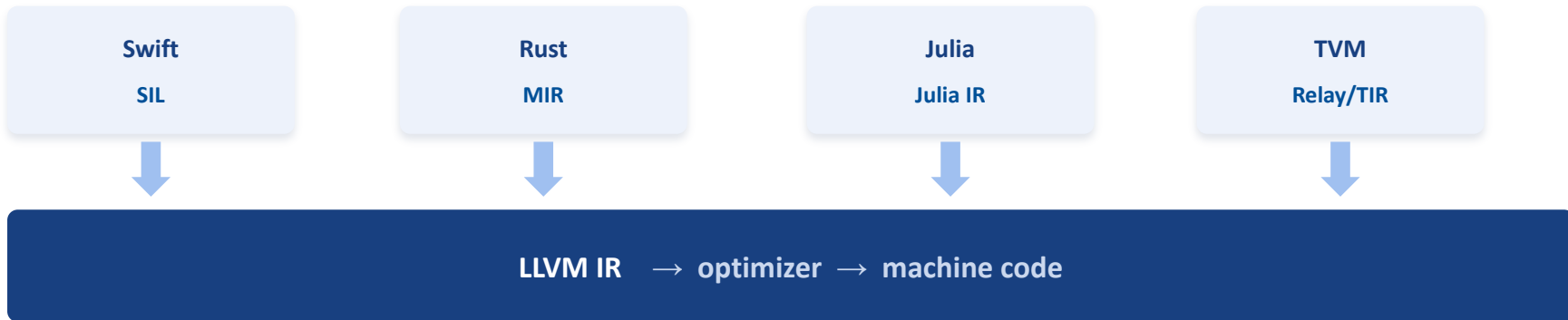
The ceiling: a single low-level IR can't express AI operation: the structure optimizers need is lost before they run.

LLVM's Three-Phase Flow: Too Low-Level for AI

LLVM IR was too low-level, so each language and framework added its own higher-level IR on top.

**Everyone Built Their Own
IR Above LLVM !!**

HIGHER-LEVEL IRs — EACH BUILT FROM SCRATCH



Same infrastructure, reinvented every time

Each IR re-implements the same plumbing from scratch: SSA data structures, a pass manager, a verifier, printer / parser, source-location tracking and test tooling. Effort is duplicated, tools are siloed, and nothing is shared across projects.

The takeaway: everyone needs custom IRs —> nobody should have to rebuild the plumbing.

MLIR(Multi-level Intermediate Representation): A Framework to Build IRs



Not another IR, infrastructure for defining your own IRs (“**dialects**”), for any domain.

You get the compiler plumbing for free

- SSA-based IR core: ops, types, attributes, regions
- Pass manager & pattern-rewrite framework
- Verifier, printer / parser
- Source-location tracking & diagnostics
- Testing & tooling infrastructure

Define a dialect, not a compiler.

Dialects for any domain

- **AI / ML**
tosa · linalg · stablehlo · torch
- **Hardware design**
CIRCT for RTL, FIRRTL, chip design
- **Languages**
Flang FIR (Fortran) · Swift-style IRs
- **Accelerators & DSLs**
custom ops & types for new chips

One framework, many IRs —> for any purpose, not just LLVM and not just AI.

The MLIR Ecosystem: Compilers Built on MLIR



One infrastructure powers language, ML and hardware compilers alike, IREE is one of many.



IREE

MLIR compiler + runtime

End-to-end: lowers models to CPU, GPU & accelerators, with a lightweight runtime (VMFB).



Triton

OpenAI · GPU kernels

Python DSL for fast GPU kernels, compiled through an MLIR-based Triton IR.



Mojo

Modular · language

AI-first language (a Python superset) built directly on MLIR for portable performance.



OpenXLA / StableHLO

Google · portability

StableHLO is an MLIR op set bridging ML frameworks to compilers across hardware.



CIRCT

hardware design

Brings MLIR to chip design — RTL, FIRRTL & SystemVerilog. MLIR beyond AI.

MLIR for AI Workloads



AI compilers define and reuse MLIR dialects to lower a model from tensors down to hardware.

PROGRESSIVE LOWERING

torch · stablehlo · tosa

High-level tensor ops

linalg

Structured, tileable ops

affine · scf · vector

Loops & vectorization

llvm

→ LLVM IR → machine code



- **Keeps high-level structure**
Fusion, tiling & layout are done at the level where the semantics still exist.
- **One flow, many targets**
Retarget the same pipeline to CPU, GPU, RISC-V & custom accelerators.
- **Reuses MLIR infrastructure**
No bespoke compiler per framework — dialects share the plumbing.
- **Ends at LLVM**
Hands off to LLVM's mature backend for final code generation.

Up next: the IREE pipeline walks exactly this ladder — Torch/StableHLO → Linalg → Flow → Stream → HAL → LLVM.

IREE Compilation Pipeline for RISC-V



What actually happens inside each of IREE's four compilation phases.



Import the Model

Torch / StableHLO

- Trace the model into a static graph (TorchDynamo / AOT export, iree-import-*).
- Convert to high-level MLIR : the Torch or StableHLO dialect.
- Ops still describe tensors: matmul, conv, gather.



Optimize & Schedule

Linalg → Flow → Stream

- Lower to Linalg; run whole-program opts (fusion, dead-code removal).
- Group ops into dispatches : independent units of work (Flow).
- Schedule dispatches to run in parallel; plan buffers (Stream).



Generate Code

Vector → LLVM dialect

- Pick tile sizes & a codegen pipeline for each dispatch.
- Tile loops and vectorize the math (RVV vectors on RISC-V).
- Lower step by step down to the LLVM dialect.



HAL & Runtime

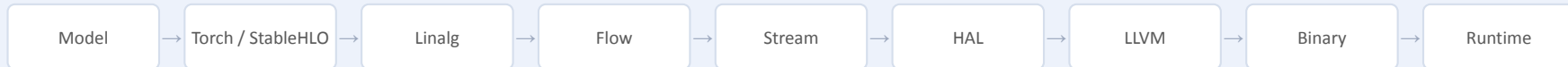
HAL → VM → VMFB

- Device code → LLVM → RISC-V binary (.so / .a).
- Host code → HAL: a portable API for any device (buffers, dispatch, sync).
- Lower HAL → VM and bundle it all into one VMFB.
- IREE runtime runs the VMFB via HAL drivers (main + workers).

Llama.cpp: The Converse Approach LLaMA C++

WHAT WE COVERED · IREE

Compiler passes generate the code



NOW · LLAMA.CPP

Hand-written C/C++ compute graph



What it is

Open-source LLM inference engine in plain C/C++ with support for 100+ model architectures.

Where it is used

Many architectures from standard CPUs like x86, Arm and RISC-V, to GPUs, to custom hardware.

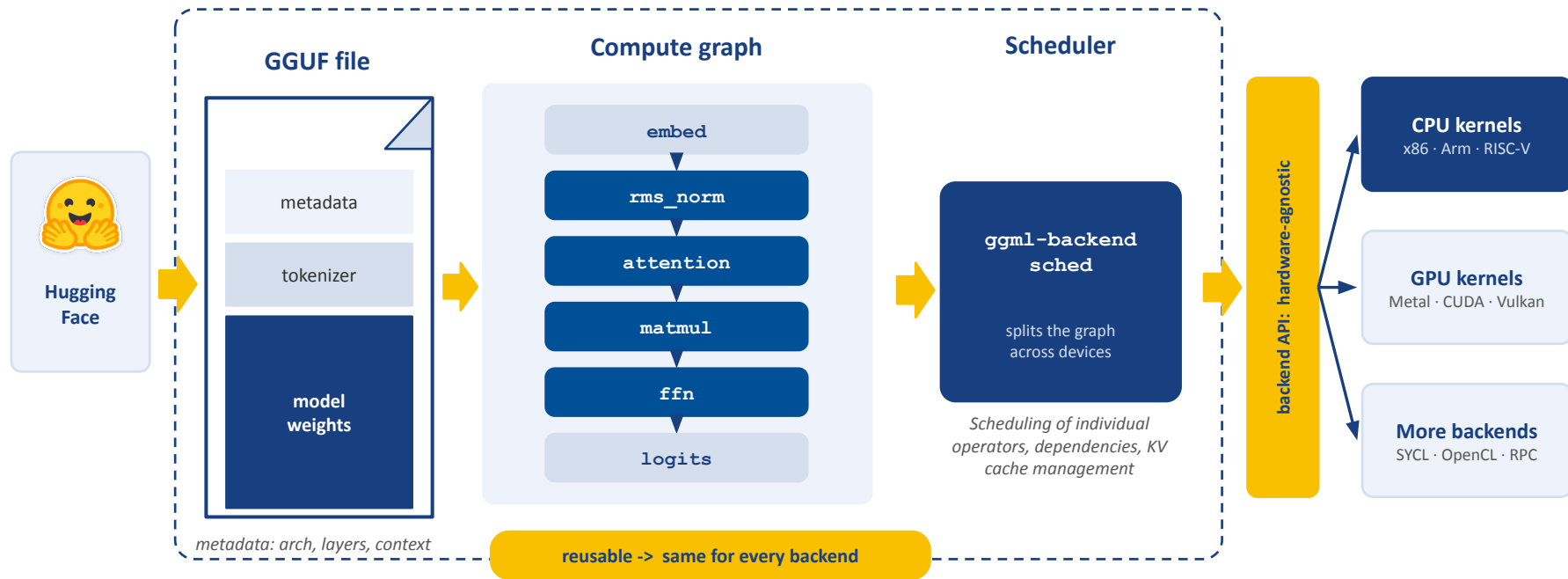
What makes it fast

Hand-written, optimized operators and kernels, and quantization formats tuned for each ISA.

*IREE's compiler approach forces dependence on frameworks like LLVM/MLIR; **Llama.cpp** allows easier onboarding with just a C/C++ runtime.*

Llama.cpp: Pipeline and Approach LLaMA C++

A simple C/C++ pipeline from model to runtime.



kernel: the small function that actually computes one graph node (a matrix multiply, a norm) on a specific device

Quantization Formats LLaMA⁺⁺

The first way llama.cpp gets its speed: make the model smaller.

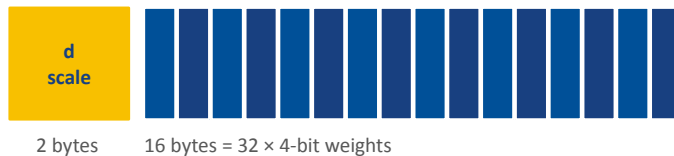
Formats in llama.cpp

Format	bits / wt	8B model	accuracy loss	speed
F16	16	~16 GB	none (baseline)	1×
Q8_0	8.5	~8.5 GB	negligible	~2×
Q4_K_M	~4.8	~4.9 GB	small (the default)	~3×
Q4_0	4.5	~4.7 GB	small	~3×
IQ2 / IQ3	2 to 3.5	~2.5–3.5 GB	noticeable	~4–5×

WHAT IS QUANTIZATION?

Storing each weight in fewer bits: a 16-bit float becomes a small integer plus a scale shared by its block.

Example: Q4_0 block structure



$$w \approx d \cdot (q - 8)$$

The scale d turns each 4-bit integer q back into a real weight: 32 weights in 18 bytes (from 64 originally)

The compromise: Smaller model size and higher performance, but at the cost of accuracy. Each hardware has its own optimal point.

Compute Kernels: Where the Speed Comes From LLaMA

The second way: hand-written optimized kernels for each operator

x86 AVX2 · AVX-512 · AMX

Arm NEON · SVE · i8mm

RISC-V RVV 1.0

Fallback portable scalar C

```
// F16 matmul, scalar reference
for (int i = 0; i < M; i++)
  for (int j = 0; j < N; j++) {
    float sum = 0.0f;
    for (int k = 0; k < K; k++)
      sum += f16_to_f32(A[i][k])
        * f16_to_f32(B[j][k]);
    C[i][j] = sum;
  }
```

```
// Same kernel, RVV 1.0 intrinsics
for (int i = 0; i < M; i++)
  for (int j = 0; j < N; j++) {
    vfloat32m2_t acc = vzero_f32m2();
    for (int k = 0; k < K; k += vl) {
      vl = vsetvl_e16m1(K - k);
      vfloat16m1_t a = vle16_f16m1(&A[i][k], vl);
      vfloat16m1_t b = vle16_f16m1(&B[j][k], vl);
      acc = vfwmaccc_f32m2(acc, a, b, vl);
    }
    C[i][j] = vreduce_sum(acc);
  }
```

vsetvl: asks the CPU how wide its vectors are;
the same binary runs on any VLEN

vfwmaccc: one instruction = 16+
multiply-accumulates, fp16 in, fp32 accumulation

The multiplier: ~30 quant formats × 4+ ISAs × dozens

Compute Kernels for RISC-V LLaMA C++

RVV lagged behind x86 and ARM for kernel support. Several engineers at 10xE contributed RVV kernels upstream.

RVV 1.0 kernels

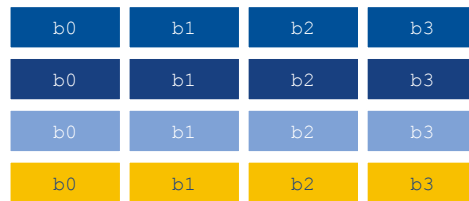
- Key operations: matmul, flash attention, softmax, RMSNorm and more.
- Key quantization types: Q4_0, Q8_0, K-quants.
- Vector-length support from 128-bit VLEN to 1024-bit.

Further optimizations: repacking

- At load, interleave the blocks of 4 or 8 weight rows into one tile.
- One vector load then feeds several output rows; scales are converted once per tile.
- The layout is chosen for the vector unit, not for the file.

Repacking, visually

Before: one row per block, rows far apart in memory



After: one tile = block b of rows 0-3, contiguous



one vector load → 4 output rows; 4 scales converted once

Comparing IREE and Llama.cpp for AI Inference

Both frameworks have their own strengths and costs.

IREE · MLIR-based Compiler



STRENGTHS

- Better coverage through reusable compiler components
- Systematic optimization: fusion, tiling, layout, scheduling
- Retargets to GPUs and custom accelerators
- Performance improves as the compiler improves

COSTS

- Long pipeline; dependency on large frameworks like MLIR/LLVM
- Peak performance still often needs hand-tuned microkernels
- New op or dialect gap = the model does not compile

Llama.cpp · C/C++ Inference Runtime LLAMA C++

STRENGTHS

- Fast on day one; readable C you can profile and step through
- Quantization formats designed hand-in-hand with kernels
- Trivial to run: download a GGUF and go
- Large community support enables faster model enablement

COSTS

- Every new op and format is manual labour
- Less reusable components
- Focused on LLMs and VLMs

Convergence: IREE has shifted towards hand-written microkernels for the paths the compiler can't beat; Llama.cpp has also built a scheduler and backend abstraction.

Thank You!

Questions?

<https://10xengineers.ai/contact-us/>